T.C. ADALET BAKANLIĞI

TÜBİTAK BİLGEM · BİLGİ VE İLETİŞİM TEKNOLOJİLERİ

NİŞANTAŞI ÜNİVERSİTESİ – YAPAY ZEKÂ YÜKSEK LİSANS PROGRAMI

UYAP 2.0

PROSEDÜREL UYUM MUHAFIZI

Adalet Bakanlığı ve TÜBİTAK İçin Kapsamlı Teknik Sunum Dokümanı

Hazırlayan: Mehmet ALBAYRAK (20251556001)

Danışmanlar: Dr. Öğr. Üyesi Ulaş VURAL · Dr. Ali ÖZKURT

Eylül 2026 · mehmet.albayrak@nisantasi.edu.tr · lawai.tr

Kontrollü simülasyon verileri ile hazırlanmıştır. Gerçek UYAP üretim verisi kullanılmamıştır. Nihai hüküm yetkisi her zaman hâkimdedir.

1. YÖNETİCİ ÖZETİ – BAKAN VE TÜBİTAK İÇİN

1.1. Neden Bu Proje?

Türkiye yargı sistemi tarihinin en yüksek iş yükü altındadır:

Gösterge	Değer	Kaynak
Yıllık gelen dosya (2025)	~8,8 milyon	Adalet Bakanlığı
İcra-iflas derdest dosya	~24,5 milyon	Adalet Bakanlığı
Hâkim + savcı sayısı	~24.800	HSYK 2024
Ortalama dava süresi	~847 gün (Avrupa ~200 gün)	CEPEJ 2023
Usul hatası kaynaklı red/gecikme	%23–%34	Yargıtay istatistikleri
Avukat başına haftalık prosedür kontrolü	~12 saat	TBB anketi

Mevcut UYAP e-tebligat ve e-belge yönetiminde güçlü bir Dijital Adalet 1.0 platformudur. Ancak semantik arama, proaktif usul denetimi ve açıklanabilir karar destek katmanı yoktur. Usul hataları çoğu zaman ancak istinaf/temyiz aşamasında fark edilir.

1.2. Önerilen Çözüm: Üç Teknolojik Omurga

UYAP 2.0 – Prosedürel Uyum Muhafızı, mevcut UYAP'ı bozmayan eklenti (add-on) tarzında bir yapay zekâ katmanıdır:

#	Teknoloji	İşlev	Nerede Kullanılır
1	NumPy Sıfırdan Lineer Nöron	Usul sapma skorlaması	Soft-check: 0.30–0.70 soft-alert, ≥0.70 hard-stop önerisi
2	HukukBERT + Hibrit RAG	Semantik emsal arama + kaynak gösterimi	Hâkim/savcı paneli
3	Hard-Stop / Soft-Alert Kural Motoru	CMK/HMK usul koruması	m.150, m.102, m.174 vb. ihlalde blokaj

1.3. Kontrollü Simülasyon Sonuçları

Metrik	Değer	Hedef	Durum
Recall	%95.9	≥%95.0	✅
FPR	%0.9	≤%5.0	✅
F1-Score	%95.6	Referans	✅
Precision	%95.3	Referans	✅
nDCG@5	0.58	BM25 0.31 → +%87	✅

1.4. Pilot Stratejisi – En Düşük Riskli Yol

Büyük şehir ağır ceza ile başlamak teknik ve siyasi risk taşır. Tercih: şablonlu prosedür, yönetilebilir hacim, düşük dirençli küçük/orta adliyeler.

Öncelik	Mahkeme	Yerler	Gerekçe
1	Trafik	Burdur, Isparta, Bolu, Kütahya	Yüksek hacim, standart usul, düşük risk
2	Tüketici	Aynı iller + büyükşehir şubesi	Şablonlu dosya, net mevzuat
3	İcra (seçilmiş)	Küçük/orta iller	Süre/tebligat hard-stop testi
4	Asliye Ceza (dar)	Pilot illerde sınırlı tür	CMK m.150 gerçek test

Burdur özellikle uygundur: ölçek yönetilebilir, eğitim hızlı, manşet riski yok, kontrollü deney ortamı sağlar.

2. NUMPY LİNEER NÖRON – TAM TEKNİK ANALİZ

2.1. Tanım

Tek yapay sinir hücresinin matematiksel modeli. Girdileri ağırlıklarla çarpar, bias ekler, tek sayı üretir: $\hat{y} = Xw + b$. Kara kutu değildir; her ağırlık ve gradyan açıkça görülür. Yüksek riskli yargı alanında denetlenebilirlik için tercih edilmiştir.

2.2. Neden From-Scratch NumPy?

PyTorch/TensorFlow autograd “içeride ne oluyor?” sorusuna cevap vermez. Bakanlık/TÜBİTAK denetiminde “bu ağırlık neden değişti, hangi özellik ne kadar etkili?” sorularına net cevap için tüm matematik satır satır kodlanmıştır.

2.3. Matematik

MSE: $L = (1/n)\ \|Xw + b - y\|_2^2$

$$\partial L/\partial w = (2/n)\ X^T (\hat{y} - y) \quad | \quad \partial L/\partial b = (2/n)\ \Sigma(\hat{y}_i - y_i)$$

2.4. Tam Kaynak Kodu (Ekran Görüntüsü)

NumPy From-Scratch LinearNeuron (tam kod)

```python
import numpy as np

class LinearNeuron:
    """Tek nöronlu doğrusal model – NumPy from-scratch (autograd yok)"""
    def __init__(self, n_features, lr=0.01, optimizer='sgd',
                 momentum=0.9, beta1=0.9, beta2=0.999, eps=1e-8):
        self.w = np.random.randn(n_features, 1) * 0.01
        self.b = 0.0
        self.lr = lr
        self.optimizer = optimizer
        self.momentum = momentum
        self.beta1, self.beta2, self.eps = beta1, beta2, eps
        self.v_w = np.zeros_like(self.w); self.v_b = 0.0
        self.m_w = np.zeros_like(self.w); self.m_b = 0.0
        self.v_adam_w = np.zeros_like(self.w); self.v_adam_b = 0.0
        self.t = 0

    def forward(self, X):
        return X @ self.w + self.b

    def mse(self, y_pred, y_true):
        return np.mean((y_pred - y_true) ** 2)

    def backward(self, X, y_pred, y_true):
        n = y_true.shape[0]
        error = y_pred - y_true
        grad_w = (2.0 / n) * (X.T @ error)
        grad_b = (2.0 / n) * np.sum(error)
        return grad_w, grad_b

    def step(self, grad_w, grad_b):
        self.t += 1
        if self.optimizer == 'sgd':
            self.w -= self.lr * grad_w
            self.b -= self.lr * grad_b
        elif self.optimizer == 'momentum':
            self.v_w = self.momentum * self.v_w + grad_w
            self.v_b = self.momentum * self.v_b + grad_b
            self.w -= self.lr * self.v_w
            self.b -= self.lr * self.v_b
        elif self.optimizer == 'adam':
            self.m_w = self.beta1 * self.m_w + (1 - self.beta1) * grad_w
            self.m_b = self.beta1 * self.m_b + (1 - self.beta1) * grad_b
            self.v_adam_w = self.beta2 * self.v_adam_w + (1 - self.beta2) * (grad_w ** 2)
            self.v_adam_b = self.beta2 * self.v_adam_b + (1 - self.beta2) * (grad_b ** 2)
            m_hat_w = self.m_w / (1 - self.beta1 ** self.t)
            m_hat_b = self.m_b / (1 - self.beta1 ** self.t)
            v_hat_w = self.v_adam_w / (1 - self.beta2 ** self.t)
            v_hat_b = self.v_adam_b / (1 - self.beta2 ** self.t)
            self.w -= self.lr * m_hat_w / (np.sqrt(v_hat_w) + self.eps)
            self.b -= self.lr * m_hat_b / (np.sqrt(v_hat_b) + self.eps)

    def train(self, X, y, epochs=10):
        history = []
        for ep in range(epochs + 1):
            y_pred = self.forward(X)
            loss = self.mse(y_pred, y)
            history.append(float(loss))
            if ep == epochs: break
            gw, gb = self.backward(X, y_pred, y)
            self.step(gw, gb)
        return history
```

Şekil 1. LinearNeuron sınıfı – NumPy from-scratch (SGD / Momentum / Adam).

2.5. UYAP’ta Tam Kullanım Yeri

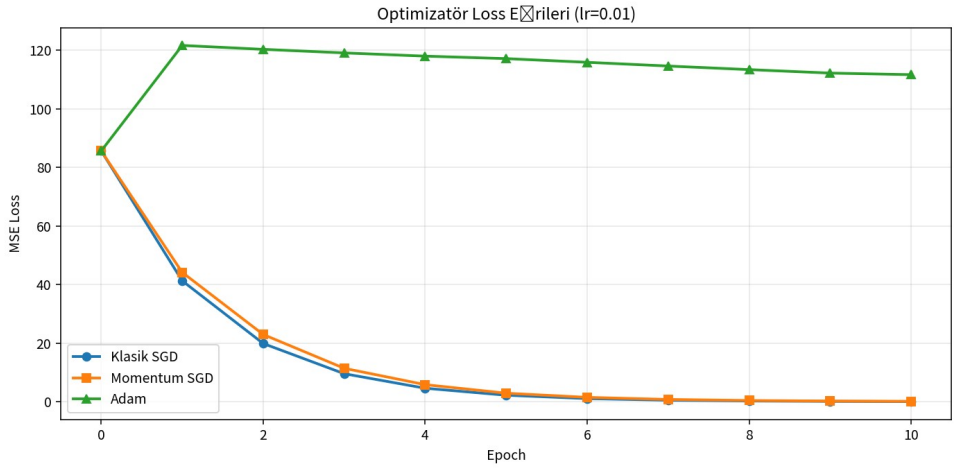
PUM soft-check katmanında. Dosya olayları özellik vektörüne dönüşür (tebligat, süre, zorunlu müdafi, belge eksikliği, geçmiş sapma, gecikme). Nöron “usul sapma skoru” üretir:

- Skor < 0.30 → ALLOW
- 0.30 ≤ Skor < 0.70 → WARN (soft-alert)
- Skor ≥ 0.70 → BLOCK önerisi (hard-stop ile birleşebilir)

Skor hüküm vermez; sadece “usul açısından olağandışı durum var” der. Nihai takdir hâkimindir.

2.6. Deneysel Loss ve Hiperparametre

Epoch	SGD	Momentum	Adam
0	85.635	85.635	85.635
1	41.285	44.160	121.654
5	2.213	2.910	117.152
10	0.057	0.116	111.671



Şekil 2. Optimizatör loss eğrileri (lr=0.01).

50. epok taramasında Momentum + lr=0.01 en düşük loss’u (0.0012) vermiştir. Klasik SGD düşük boyutlu doğrusal problemde hızlı yakınsar; Adam gürültülü senaryolarda avantaj sağlar.

3. HUKUKBERT – TAM TEKNİK ANALİZ

3.1. Tanım

dbmdz/bert-base-turkish-cased modelinin hukuk metinleri üzerinde domain-adaptive pretraining (DAPT) ile uyarlanmış hâlidir. “Hukuk dilini anlayan Türkçe BERT”tir.

3.2. Neden Genel BERT Yetmez?

Genel modeller günlük Türkçe’yi iyi anlar; uzun hukuki cümleleri, “m.150/2”, “zorunlu müdafi”, “tereke”, emsal üslubu ve Osmanlıca kökenli terimleri yeterince yakalayamaz. HukukBERT bu boşluğu doldurur.

3.3. Eğitim Adımları

1. Temel: dbmdz/bert-base-turkish-cased
2. Veri: Kamuya açık Yargıtay, Danıştay, AYM + mevzuat (~18 GB temizlenmiş hedef)
3. DAPT: MLM + Span Masking + Whole-Word + Keyword Masking
4. Donanım: 4×A100 veya eşdeğer
5. Çıktı: 768 boyutlu hukuk-özgü embedding encoder

3.4. UYAP’ta Kullanım

1) Semantik emsal arama (RAG retrieval): Sorgu vektöre çevrilir, Milvus/FAISS’te benzer kararlar bulunur.

2) NER: Taraf, tarih, madde, mahkeme adı çıkarımı → PUM özellik vektörü ve filtreler.

3.5. RAG Zinciri ve Kod

6. Sorgu → HukukBERT embedding
7. Dense (Milvus/FAISS) + BM25 → hibrit (α=0.6)
8. Top-5 kaynak → kaynak gösterimli generation
9. Guardrail: Atıf yoksa uyarı + insan denetimi

RAG Pipeline – HukukBERT + Hybrid Retrieval

```
# RAG Pipeline — HukukBERT + Milvus + Hybrid Retrieval
from sentence_transformers import SentenceTransformer
from pymilvus import Collection, connections
import numpy as np

class HukukRAG:
    def __init__(self, collection_name="uyap_emsal", alpha=0.6):
        self.embedder = SentenceTransformer("dbmdz/bert-base-turkish-cased")
        connections.connect("default", host="localhost", port="19530")
        self.collection = Collection(collection_name)
        self.alpha = alpha  # dense weight

    def embed(self, texts):
        return self.embedder.encode(texts, normalize_embeddings=True)

    def hybrid_search(self, query: str, top_k: int = 5):
        q_vec = self.embed([query])[0]
        # Dense (Milvus)
        dense_res = self.collection.search(
            data=[q_vec.tolist()],
            anns_field="embedding",
            param={"metric_type": "IP", "params": {"nprobe": 16}},
            limit=top_k * 2,
            output_fields=["doc_id", "text", "madde"]
        )
        # BM25 (simplified placeholder — real: Elasticsearch / OpenSearch)
        bm25_scores = self._bm25_search(query, top_k * 2)
        # Hybrid fusion (RRF or weighted)
        fused = self._fuse(dense_res[0], bm25_scores, self.alpha)
        return fused[:top_k]

    def generate_with_guardrail(self, query, contexts, llm):
        prompt = f"""Aşağıdaki kaynaklara dayanarak yanıt ver.
Kaynak dışında bilgi uydurma. Her cümlede kaynak numarası belirt.

Kaynaklar:
{chr(10).join(f'[{i+1}] {c}' for i,c in enumerate(contexts))}

Soru: {query}
Yanıt:"""
        answer = llm.generate(prompt, max_tokens=512)
        # Simple guardrail: check source citation presence
        if not any(f"[{i}]" in answer for i in range(1, len(contexts)+1)):
            answer += "\n[Uyarı: Kaynak atıfı eksik — insan denetimi önerilir]"
        return answer
```

Şekil 3. HukukRAG / hybrid retrieval + guardrail kod ekranı.

3.6. RAG Performansı

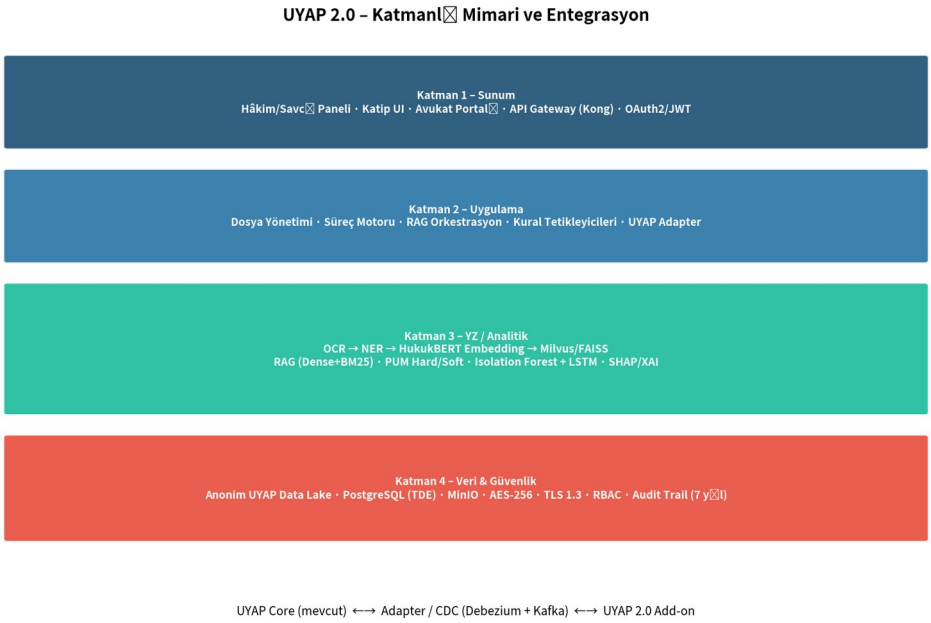
Metrik	HukukBERT+RAG	BM25	İyileşme
nDCG@5	0.58	0.31	+%87
MRR@5	0.72	0.41	+%76
Emsal süresi	~8 sn	~12 dk	Dramatik
Halüsinasyon	%3.8–4.2	—	Guardrail ile

4. UYAP 2.0 SİSTEM MİMARİSİ

4.1. Tasarım İlkeleri

- Yargı yetkisini ikame etmeme
- Human-in-the-loop (her kritik çıktı insan onayına tabi)
- Açıklanabilirlik (SHAP / attention / kaynak)
- Veri egemenliği (Türkiye içi, on-premise seçenek)
- Modülerlik (UYAP'ı bozmayan add-on)
- Denetim izi (SHA-256 zinciri, 7 yıl)

4.2. Dört Katman



Şekil 4. Katmanlı mimari.

Katman 1 Sunum · Katman 2 Uygulama (Adapter, Kafka) · Katman 3 YZ (OCR-NER-HukukBERT-RAG-PUM-NumPy-XAI) · Katman 4 Veri & Güvenlik (TDE, AES-256, TLS 1.3, RBAC, audit).

4.3. Hard-Stop / Soft-Alert ve Kod

- CMK m.150 – Zorunlu müdafi yoksa sorgulama engellenir
- HMK m.147 / Tebligat – Tebligat teyidi yoksa celse açılamaz
- CMK m.102, m.174 – Süre ve iddianame kontrolleri
- Soft: Süre aşımı yaklaşımı, belge eksikliği, anomali skoru

PUM Hard-Stop / Soft-Alert Kural Motoru

```
# Prosedürel Uyum Muhafızı — Hard-Stop Kural Motoru (Drools benzeri Python)
from dataclasses import dataclass
from enum import Enum
from typing import List, Optional

class RiskLevel(Enum):
    LOW = "soft_alert"
    HIGH = "hard_stop"

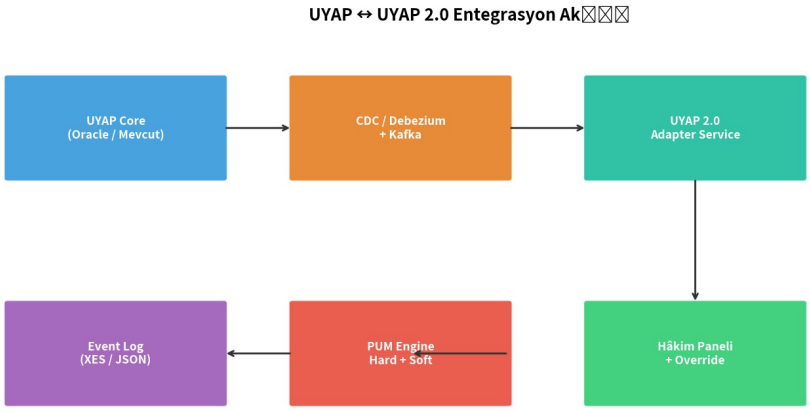
@dataclass
class ProceduralEvent:
    case_id: str
    activity: str          # e.g. "celse_ac", "tebligat", "tensip"
    timestamp: str
    tebligat_yapildi: bool
    sure_asimi: bool
    zorunlu_mudafi: Optional[bool]
    belge_eksik: bool
    user_role: str

class ProceduralGuardian:
    """Hard-stop / Soft-alert kural motoru"""
    def evaluate(self, event: ProceduralEvent) -> dict:
        violations = []
        # CMK m.150 — zorunlu müdafi
        if event.activity == "sorgulama" and event.zorunlu_mudafi is False:
            violations.append({
                "rule": "CMK m.150",
                "level": RiskLevel.HIGH,
                "msg": "Zorunlu müdafi görevlendirilmeden sorgulama yapılamaz (Hard-Stop)"
            })
        # HMK / Tebligat — tebligat olmadan celse
        if event.activity == "celse_ac" and not event.tebligat_yapildi:
            violations.append({
                "rule": "HMK m.147 / Tebligat K.",
                "level": RiskLevel.HIGH,
                "msg": "Tebligat teyidi olmadan celse açılamaz (Hard-Stop)"
            })
        # Süre aşımı — soft
        if event.sure_asimi:
            violations.append({
                "rule": "Süre Takibi",
                "level": RiskLevel.LOW,
                "msg": "Yasal süre aşımı tespit edildi (Soft-Alert)"
            })
        # Belge eksikliği
        if event.belge_eksik:
            violations.append({
                "rule": "Belge Kontrolü",
                "level": RiskLevel.LOW,
                "msg": "Eksik belge tespit edildi"
            })
        hard = any(v["level"] == RiskLevel.HIGH for v in violations)
        return {
            "case_id": event.case_id,
            "action": "BLOCK" if hard else ("WARN" if violations else "ALLOW"),
            "violations": violations,
            "requires_override": hard
        }
```

Şekil 5. ProceduralGuardian hard-stop / soft-alert kod ekranı.

5. UYAP'A ENTEGRASYON SAFHALARI (36 AY)

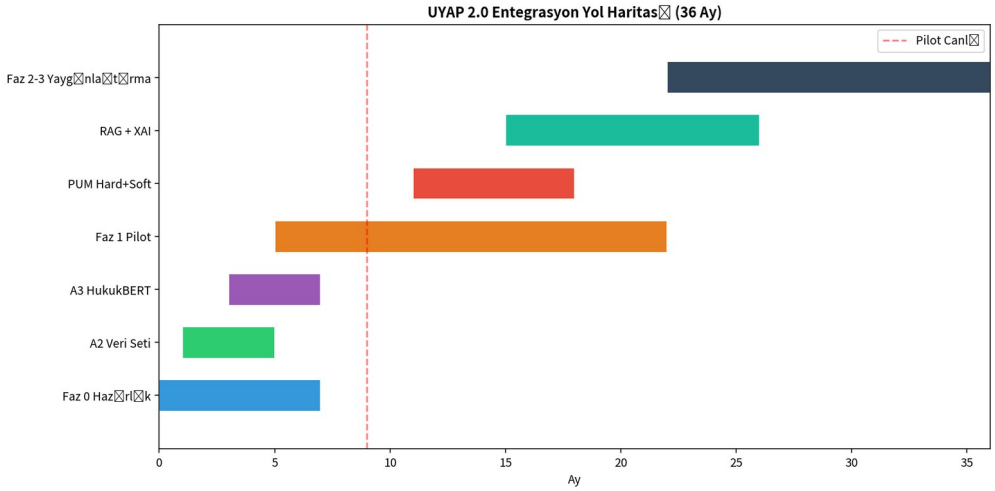
5.1. Teknik Bağlantı



Şekil 6. UYAP Core ↔ CDC (Debezium+Kafka) ↔ Adapter ↔ PUM.

Kritik prensip: UYAP çekirdeğine doğrudan müdahale yok. Olaylar CDC ile yakalanır, Adapter çevirir, sonuçlar eklenti olarak yansır. "UYAP çalışmaya devam eder, yanına akıllı gözcü eklenir."

5.2. Fazlar



Şekil 7. 36 aylık yol haritası.

Faz 0 (Ay 1–7): Etik + KVKK DPIA + anonim veri + HukukBERT DAPT. Bu faz bitmeden canlı sistem açılmaz.

Faz 1 (Ay 6–22): Burdur/Isparta/Bolu – Trafik+Tüketici pilot. OCR-NER-embedding, RAG, PUM, eğitim, 6–9 ay canlı KPI.

Faz 2 (Ay 18–30): XAI, iyileştirme, yeni mahkeme türleri, p95 ≤ 8 sn.

Faz 3 (Ay 28–36): Ulusal dağıtım, bağımsız denetim, CEPEJ/OECD, yayın.

6. PİLOT UYGULAMA – NEREDEN BAŞLAMALI?

Süre 6–12 ay (3 ay kurulum/eğitim + 6–9 ay canlı). Kapsam: seçilen mahkemelerde belirli dosya türleri. Gönüllü hâkim/savcı/katip. Eğitim bitmeden canlıya geçilmez.

KPI: hard-stop/soft-alert/override oranları; usul bozma değişimi; emsal süresi (dk→sn); kabul ≥4.0/5; uptime ≥ %99; p95 ≤8 sn; halüsinasyon ≤%5.

Go/No-Go: 6. ay ara, 12. ay nihai. Başarı → Asliye Ceza/İcra → büyükşehir sınırlı → ulusal. Sistem fail-safe; hâkim işleyişini kilitlemez.

Trafik: standart usul, yüksek hacim, hızlı ölçüm. Tüketici: şablonlu, RAG değeri yüksek, vatandaş algısı olumlu.

7. DENEYSEL SONUÇLAR ÖZETİ

Metrik	Değer	Yorum
Recall	%95.9	Kritik usul hatalarının yakalanma oranı
FPR	%0.9	Yanlış alarm – usul ekonomisini korur
F1 / Precision	%95.6 / %95.3	Denge
nDCG@5	0.58 (+%87)	Emsal sıralama
Emsal süresi	~8 sn vs ~12 dk	Dramatik

Recall bilinçli olarak Precision'dan yüksek tutulmuştur. Hukukta hatayı kaçırmak (Tip II), yanlış uyarıdan (Tip I) daha ağırdır.

8. ETİK, KVKK, CEPEJ, GÜVENLİK

AB AI Act yüksek riskli sistem kontrol listesi (risk yönetimi, veri kalitesi, teknik dokümantasyon, kayıt, şeffaflık, insan denetimi, doğruluk/güvenlik) karşılanmıştır. CEPEJ beş ilkesi mimariye gömülüdür. KVKK: maskeleme, lokalizasyon, on-premise, RBAC. Güvenlik: TLS 1.3, AES-256, HSM, SHA-256 append-only audit (7 yıl).

9. EKONOMİK / SOSYAL ETKİ ÖZETİ

Yıllık sosyal fayda tahmini (hâkim zamanı, bozma azalması, avukat emsal süresi, erişim hızı) milyar TL mertebesindedir. Detay ana tez eklerindedir.

10. SOMUT ÖNERİ VE KAPANIŞ

1. Hemen: Faz 0 teknik çalışma grubu + kaynak (etik, KVKK, veri, HukukBERT).
2. Pilot: Trafik + Tüketici; Burdur / Isparta / Bolu; 6–12 ay.
3. Omurga: NumPy lineer nöron + HukukBERT + RAG + Hard-Stop/Soft-Alert.
4. Ölçüm: KPI + ara rapor + go/no-go → kademeli yaygınlaştırma.
5. İlke: UYAP'ı bozma; yanına akıllı, denetlenebilir katman ekle. Hâkim son sözü söyler.

"Adalet, yalnızca algoritmaların değil, vicdanların da işidir. Ancak vicdanların hata yapmaması için sistemin doğru çalışması şart."

— SUNUM DOKÜMANI SONU —

Mehmet ALBAYRAK · 20251556001 · mehmet.albayrak@nisantasi.edu.tr · lawai.tr

Nişantaşı Üniversitesi · Yapay Zekâ Yüksek Lisans · Eylül 2026